

Prezentarea 2

Prof. Coordonator: Ș.I.dr.ing.Stelian-Nicolae NICOLAE

Mentor: Pătrașcu Hary

Să adăugăm mai multe animații

Asset pack-ul pe care îl folosim are animațiile separate pentru tool-uri, așadar va trebui să mai adăugăm un nod AnimatedSprite2D player-ului.

Redenumiți nodurile AnimatedSprite2D pentru a le putea diferenția mai ușor.

Animațiile pentru unelte(tool-uri) încep cu numele 'tools'

Adăugați animațiile pentru ambele animated sprites:

- Run
- Walk
- Idle
- Dig
- Water

▼  Player

 CharacterAnimatedSprite

 ToolsAnimatedSprite

 CollisionShape2D

 Camera2D

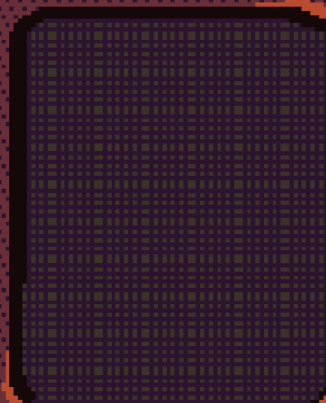


Flip sprite în funcție de direcția de mers

Implementarea curentă a script-ului nostru nu ține cont de direcția de mers a jucătorului, astfel sprite-ul 'se uită' în dreapta chiar dacă merge în stânga.

Exercițiu: De ce credeți că avem nevoie ca să dăm flip la sprite?

- memorarea direcției curente(o variabilă)
- o proprietate a sprite-ului pe care să o putem modifica să îi dea flip



Care credeți că este problema la acest cod?

```
if _input_dir.x > 0:  
    _facing_right = true  
else:  
    _facing_right = false
```

```
animated_sprite.flip_h = not _facing_right
```

Două modalități de a rezolva această problemă

```
if _input_dir.x > 0:  
    >| _facing_right = true  
elif _input_dir.x < 0:  
    >| _facing_right = false
```

```
>| if _input_dir.x != 0:  
    >| _facing_right = _input_dir.x > 0
```

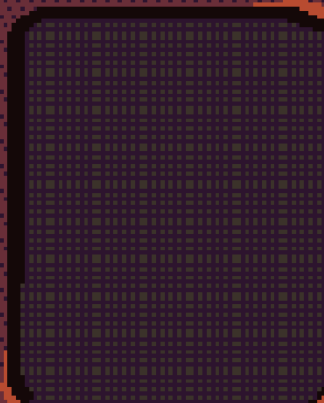
Să adăugăm mai multe stări

Am adăugat mai multe tipuri de animații, acum trebuie să scriem logica în cod pentru a le apela în funcție de starea curentă a jucătorului.

Să începem cu starea de alergat(sprint).

Exercițiu: Ce credeți că am avea nevoie pentru a implementa această stare?

- o nouă variabilă pentru viteza de sprint
- o nouă variabilă pentru a stoca viteza curentă a jucătorului(viteza de mers vs viteza de alergare)
- o nouă acțiune de input
- o variabilă pentru a stoca dacă jucătorul aleargă



O modalitate de a scrie această logică

```
_is_running = Input.is_action_pressed("sprint")
if _is_running:
>|   _current_speed = _sprint_speed
else:
>|   _current_speed = _speed|
```

Cealaltă modalitate este de a folosi operatori ternari(if pe aceeași linie în care asignăm valoarea nouă unei variabile):

```
_is_running = Input.is_action_pressed("sprint")  
_current_speed = _sprint_speed if _is_running else _speed
```



Să redenumim variabila `'_speed'` în `'_walk_speed'` pentru a fi mai sugestivă. Dacă dăm double click pe variabilă și apăsăm combinația de taste CTRL+R(de la rename) putem redenumi variabile sau alte nume care se repetă în scripturile noastre. Pentru a redenumi exclusiv cuvântul selectat, bifați setarea whole words.

1. **Introduction**
 2. **Background**
 3. **Methodology**
 4. **Results**
 5. **Conclusion**
 6. **References**
 7. **Appendix**
 8. **Index**
 9. **Table of Contents**
 10. **Summary**
 11. **Abstract**
 12. **Keywords**
 13. **Subject Headings**
 14. **Notes**
 15. **References**
 16. **Appendix**
 17. **Index**
 18. **Table of Contents**
 19. **Summary**
 20. **Abstract**
 21. **Keywords**
 22. **Subject Headings**
 23. **Notes**
 24. **References**
 25. **Appendix**
 26. **Index**
 27. **Table of Contents**
 28. **Summary**
 29. **Abstract**
 30. **Keywords**
 31. **Subject Headings**
 32. **Notes**
 33. **References**
 34. **Appendix**
 35. **Index**
 36. **Table of Contents**
 37. **Summary**
 38. **Abstract**
 39. **Keywords**
 40. **Subject Headings**
 41. **Notes**
 42. **References**
 43. **Appendix**
 44. **Index**
 45. **Table of Contents**
 46. **Summary**
 47. **Abstract**
 48. **Keywords**
 49. **Subject Headings**
 50. **Notes**
 51. **References**
 52. **Appendix**
 53. **Index**
 54. **Table of Contents**
 55. **Summary**
 56. **Abstract**
 57. **Keywords**
 58. **Subject Headings**
 59. **Notes**
 60. **References**
 61. **Appendix**
 62. **Index**
 63. **Table of Contents**
 64. **Summary**
 65. **Abstract**
 66. **Keywords**
 67. **Subject Headings**
 68. **Notes**
 69. **References**
 70. **Appendix**
 71. **Index**
 72. **Table of Contents**
 73. **Summary**
 74. **Abstract**
 75. **Keywords**
 76. **Subject Headings**
 77. **Notes**
 78. **References**
 79. **Appendix**
 80. **Index**
 81. **Table of Contents**
 82. **Summary**
 83. **Abstract**
 84. **Keywords**
 85. **Subject Headings**
 86. **Notes**
 87. **References**
 88. **Appendix**
 89. **Index**
 90. **Table of Contents**
 91. **Summary**
 92. **Abstract**
 93. **Keywords**
 94. **Subject Headings**
 95. **Notes**
 96. **References**
 97. **Appendix**
 98. **Index**
 99. **Table of Contents**
 100. **Summary**
 101. **Abstract**
 102. **Keywords**
 103. **Subject Headings**
 104. **Notes**
 105. **References**
 106. **Appendix**
 107. **Index**
 108. **Table of Contents**
 109. **Summary**
 110. **Abstract**
 111. **Keywords**
 112. **Subject Headings**
 113. **Notes**
 114. **References**
 115. **Appendix**
 116. **Index**
 117. **Table of Contents**
 118. **Summary**
 119. **Abstract**
 120. **Keywords**
 121. **Subject Headings**
 122. **Notes**
 123. **References**
 124. **Appendix**
 125. **Index**
 126. **Table of Contents**
 127. **Summary**
 128. **Abstract**
 129. **Keywords**
 130. **Subject Headings**
 131. **Notes**
 132. **References**
 133. **Appendix**
 134. **Index**
 135. **Table of Contents**
 136. **Summary**
 137. **Abstract**
 138. **Keywords**
 139. **Subject Headings**
 140. **Notes**
 141. **References**
 142. **Appendix**
 143. **Index**
 144. **Table of Contents**
 145. **Summary**
 146. **Abstract**
 147. **Keywords**
 148. **Subject Headings**
 149. **Notes**
 150. **References**
 151. **Appendix**
 152. **Index**
 153. **Table of Contents**
 154. **Summary**
 155. **Abstract**
 156. **Keywords**
 157. **Subject Headings**
 158. **Notes**
 159. **References**
 160. **Appendix**
 161. **Index**
 162. **Table of Contents**
 163. **Summary**
 164. **Abstract**
 165. **Keywords**
 166. **Subject Headings**
 167. **Notes**
 168. **References**
 169. **Appendix**
 170. **Index**
 171. **Table of Contents**
 172. **Summary**
 173. **Abstract**
 174. **Keywords**
 175. **Subject Headings**
 176. **Notes**
 177. **References**
 178. **Appendix**
 179. **Index**
 180. **Table of Contents**
 181. **Summary**
 182. **Abstract**
 183. **Keywords**
 184. **Subject Headings**
 185. **Notes**
 186. **References**
 187. **Appendix**
 188. **Index**
 189. **Table of Contents**
 190. **Summary**
 191. **Abstract**
 192. **Keywords**
 193. **Subject Headings**
 194. **Notes**
 195. **References**
 196. **Appendix**
 197. **Index**
 198. **Table of Contents**
 199. **Summary**
 200. **Abstract**
 201. **Keywords**
 202. **Subject Headings**
 203. **Notes**
 204. **References**
 205. **Appendix**
 206. **Index**
 207. **Table of Contents**
 208. **Summary**
 209. **Abstract**
 210. **Keywords**
 211. **Subject Headings**
 212. **Notes**
 213. **References**
 214. **Appendix**
 215. **Index**
 216. **Table of Contents**
 217. **Summary**
 218. **Abstract**
 219. **Keywords**
 220. **Subject Headings**
 221. **Notes**
 222. **References**
 223. **Appendix**
 224. **Index**
 225. **Table of Contents**
 226. **Summary**
 227. **Abstract**
 228. **Keywords**
 229. **Subject Headings**
 230. **Notes**
 231. **References**
 232. **Appendix**
 233. **Index**
 234. **Table of Contents**
 235. **Summary**
 236. **Abstract**
 237. **Keywords**
 238. **Subject Headings**
 239. **Notes**
 240. **References**
 241. **Appendix**
 242. **Index**
 243. **Table of Contents**
 244. **Summary**
 245. **Abstract**
 246. **Keywords**
 247. **Subject Headings**
 248. **Notes**
 249. **References**
 250. **Appendix**
 251. **Index**
 252. **Table of Contents**
 253. **Summary**
 254. **Abstract**

`_speed` No match ⬆ ⬇ Match Case ☒ Whole Words

`_walk_speed` Replace Replace All Selection Only

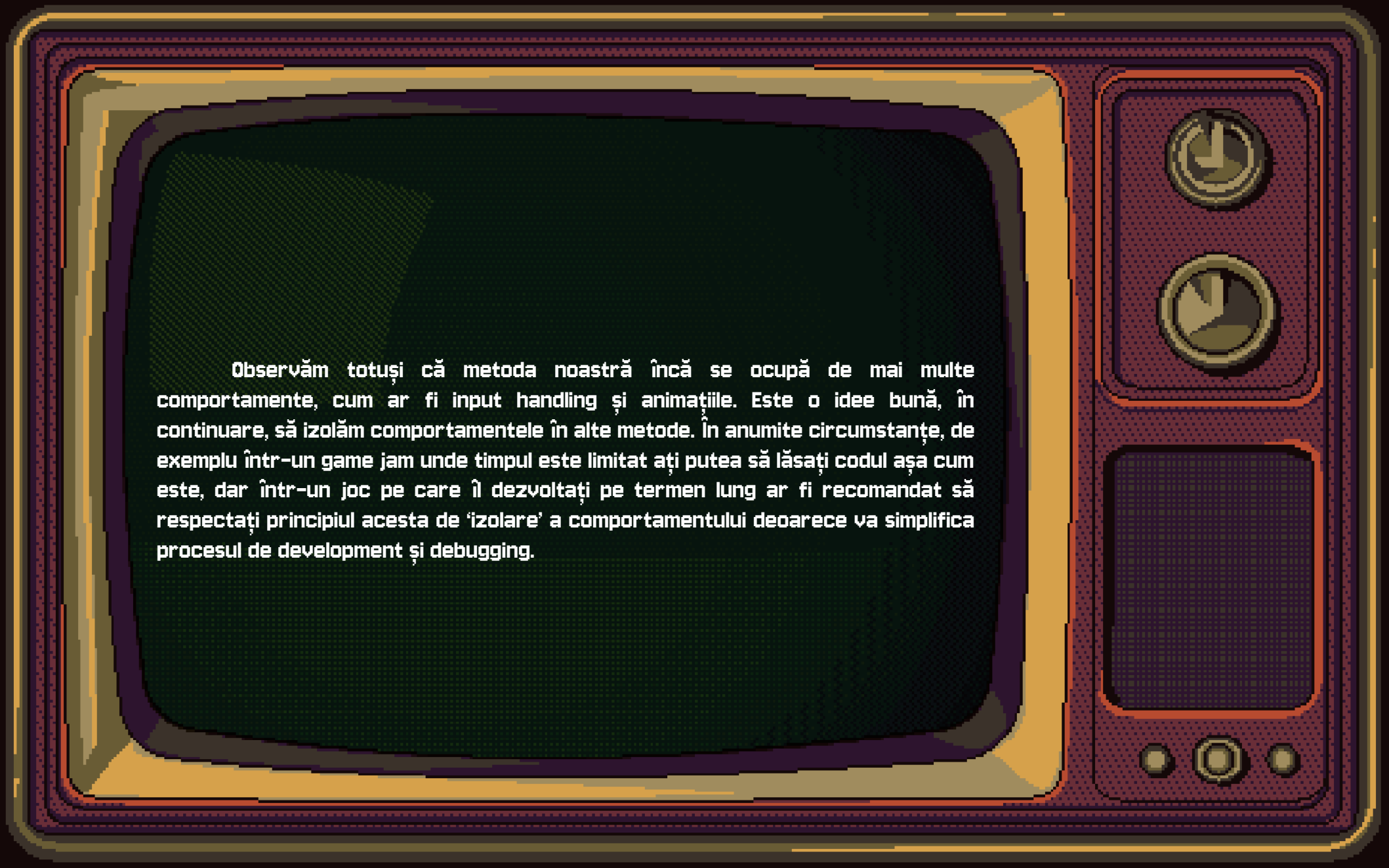
Funcții(metode)

Este o idee bună să grupăm anumite comportamente în metode(funcții), de exemplu până acum am implementat funcționalitatea de movement, așadar codul pe care îl avem îl putem pune într-o metodă care se ocupă exclusiv de acest comportament(movevement)

Exercițiu: Creați o metodă `_player_move` care nu primește niciun argument și nu returnează nimic. Apelați metoda creată în `_physics_process(delta: float)`

```
func _physics_process(delta: float) -> void:
>| _player_move()
>| move_and_slide()

func _player_move() -> void:
>| # preluam input de la tastatura -> returneaza un vector in 2D
>| _input_dir = Input.get_vector("left", "right", "up", "down")
>|
>| _is_running = Input.is_action_pressed("sprint")
>| _current_speed = _sprint_speed if _is_running else _walk_speed
>|
>| if _input_dir == Vector2.ZERO:
>| >| animated_sprite.play("idle")
>| elif _is_running:
>| >| animated_sprite.play("run")
>| else:
>| >| animated_sprite.play("walk")
>|
>| animated_sprite.flip_h = _facing_left
>| # inmultim de data asta cu variabila _current_speed pentru a ne adapta la s
>| velocity = _input_dir * _walk_speed
>|
>| if _input_dir.x != 0:
>| >| _facing_left = _input_dir.x < 0
```



Observăm totuși că metoda noastră încă se ocupă de mai multe comportamente, cum ar fi input handling și animațiile. Este o idee bună, în continuare, să izolăm comportamentele în alte metode. În anumite circumstanțe, de exemplu într-un game jam unde timpul este limitat ați putea să lăsați codul așa cum este, dar într-un joc pe care îl dezvoltați pe termen lung ar fi recomandat să respectați principiul acesta de 'izolare' a comportamentului deoarece va simplifica procesul de development și debugging.

Așa cum am discutat în cursul anterior, Godot dispune de metode built-in pe care le putem suprascrive cu comportamentul nostru custom. Așadar, să începem prin a izola input-ul de la player folosind metoda `_input`.

Exercițiu: Mutați codul care folosește clasa `'Input'` în metoda `_input`.

Hint: semnătura metodei este `_input(event: InputEvent) → void`

Dacă scrieți func `_input`, editor-ul vă va da opțiuni de autocomplete pe care le puteți selecta cu săgețile de pe tastatură și pentru a alege auto completarea pe care o vreți apăsați TAB.

```
19 ▼ func _input(event: InputEvent) -> void:
20   » # preluam input de la tastatura -> returneaza un vector in 2D
21   » _input_dir = Input.get_vector("left", "right", "up", "down")
22   » _is_running = Input.is_action_pressed("sprint")
23
24 ▼ func _player_move() -> void:
25   » _current_speed = _sprint_speed if _is_running else _walk_speed
26   »
27 ▼ » if _input_dir == Vector2.ZERO:
28   »   » animated_sprite.play("idle")
29 ▼ » elif _is_running:
30   »   » animated_sprite.play("run")
31 ▼ » else:
32   »   » animated_sprite.play("walk")
33   »
34   » animated_sprite.flip_h = _facing_left
35   » # inmultim de data asta cu variabila _current_speed pentru a ne adapta
36   » velocity = _input_dir * _walk_speed
```



Singura funcționalitate care mai trebuie izolată sunt animațiile care țin de partea vizuală a clasei Player. Din nou, pentru partea vizuală vom folosi o metodă built-in, `_process`. Metoda `_process` ar trebui utilizată pentru update-uri la părțile vizuale ale jocului, animații, sprite-uri, interfețe grafice(GUI), etc.

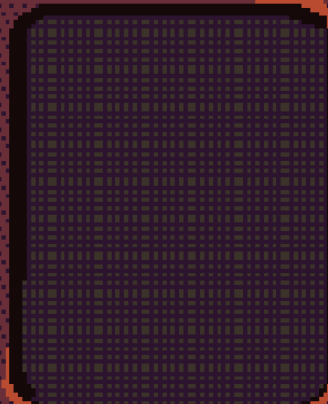
Exercițiu: Creați o metodă `_player_animate()` fără parametri, care nu returnează nimic și care va conține logica de animație și flip.

Apelați metoda creată din `_process()`

```

14  func _physics_process(delta: float) -> void:
15      _player_move()
16      move_and_slide()
17
18  func _process(delta: float) -> void:
19      _player_animate()
20
21  func _input(event: InputEvent) -> void:
22      # preluam input de la tastatura -> returneaza un vector in 2D
23      _input_dir = Input.get_vector("left", "right", "up", "down")
24      _is_running = Input.is_action_pressed("sprint")
25
26  func _player_animate() -> void:
27      if _input_dir == Vector2.ZERO:
28          animated_sprite.play("idle")
29      elif _is_running:
30          animated_sprite.play("run")
31      else:
32          animated_sprite.play("walk")
33
34      animated_sprite.flip_h = _facing_left
35
36  func _player_move() -> void:
37      _current_speed = _sprint_speed if _is_running else _walk_speed
38      # inmultim de data asta cu variabila _current_speed pentru a ne adapta la starea
39      velocity = _input_dir * _walk_speed
40
41      if _input_dir.x != 0:
42          _facing_left = _input_dir.x < 0
43

```



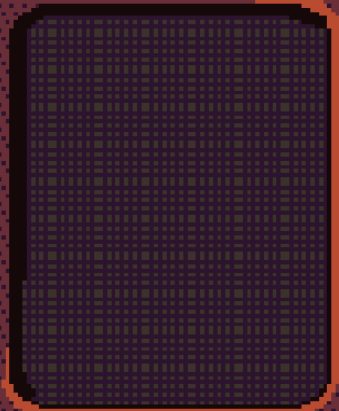


Poate pare redundant să facem toate acestea, dar pe parcurs va fi de imens ajutor cu cât adăugăm mai multă funcționalitate clasei Player. Există totuși un avantaj încă de la început, dacă lucrăm în echipă, ceilalți coechipieri le va fi mult mai ușor să înțeleagă ce face codul. De asemenea codul este self-documenting (numele metodelor și numele variabilelor practic explică deja ce face fiecare metodă/comportament), astfel nu mai e nevoie neapărat de comentarii.



În GDScript și C# există de asemenea și code regions care sunt o modalitate de documentare/ grupare a codului. Este mai mult un aspect estetic dar ajută la găsirea codului mai repede.

```
4  @export var _sprint_speed: int = 100
5  var _current_speed: int = _walk_speed
6  var _is_running: bool = false
7  var _facing_left: bool = false
8
9
10 @export var animated_sprite: AnimatedSprite2D
11 var _input_dir: Vector2 = Vector2.ZERO
12
13 #region GODOT BUILT-IN METHODS
14
15
16
17
18
19
20
21
22
23
24
25
26
27 #region PLAYER METHODS
28 func _player_animate() -> void:
29     if _input_dir == Vector2.ZERO:
30         animated_sprite.play("idle")
31     elif _is_running:
32         animated_sprite.play("run")
33     else:
34         animated_sprite.play("walk")
35
36     animated_sprite.flip_h = _facing_left
37
38 func _player_move() -> void:
39     _current_speed = _sprint_speed if _is_running else _walk_speed
40     # inmultim de data asta cu variabila _current_speed pentru a ne adapta la starea
41     velocity = _input_dir * _walk_speed
42
43     if _input_dir.x != 0:
44         _facing_left = _input_dir.x < 0
45 #endregion
46
```





Acum că am structurat codul, să adăugăm o altă funcționalitate vizuală, animated sprite-ul pentru tool-uri.

Exercițiu: Adăugați o referință la nodul 'ToolsAnimatedSprite' și dați play la animația corespunzătoare în funcție de starea player-ului.

```

3  @export var _walk_speed: int = 75
4  @export var _sprint_speed: int = 100
5  var _current_speed: int = _walk_speed
6  var _is_running: bool = false
7  var _facing_left: bool = false
8
9
10 @export var animated_sprite: AnimatedSprite2D
11 @onready var tools_animated_sprite: AnimatedSprite2D = $ToolsAnimatedSprite
12
13 var _input_dir: Vector2 = Vector2.ZERO
14
15 #region GODOT BUILT-IN METHODS
16
17
18
19 #region PLAYER METHODS
20
21 func _player_animate() -> void:
22     if _input_dir == Vector2.ZERO:
23         animated_sprite.play("idle")
24         tools_animated_sprite.play("idle")
25     elif _is_running:
26         animated_sprite.play("run")
27         tools_animated_sprite.play("run")
28     else:
29         animated_sprite.play("walk")
30         tools_animated_sprite.play("walk")
31
32     animated_sprite.flip_h = _facing_left
33
34 func _player_move() -> void:
35
36 #endregion

```

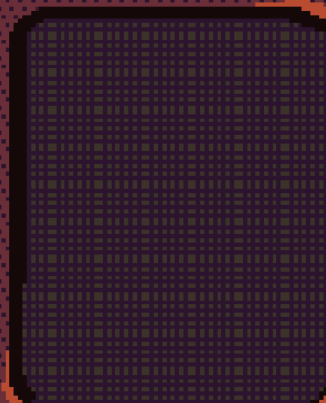
Tool-uri

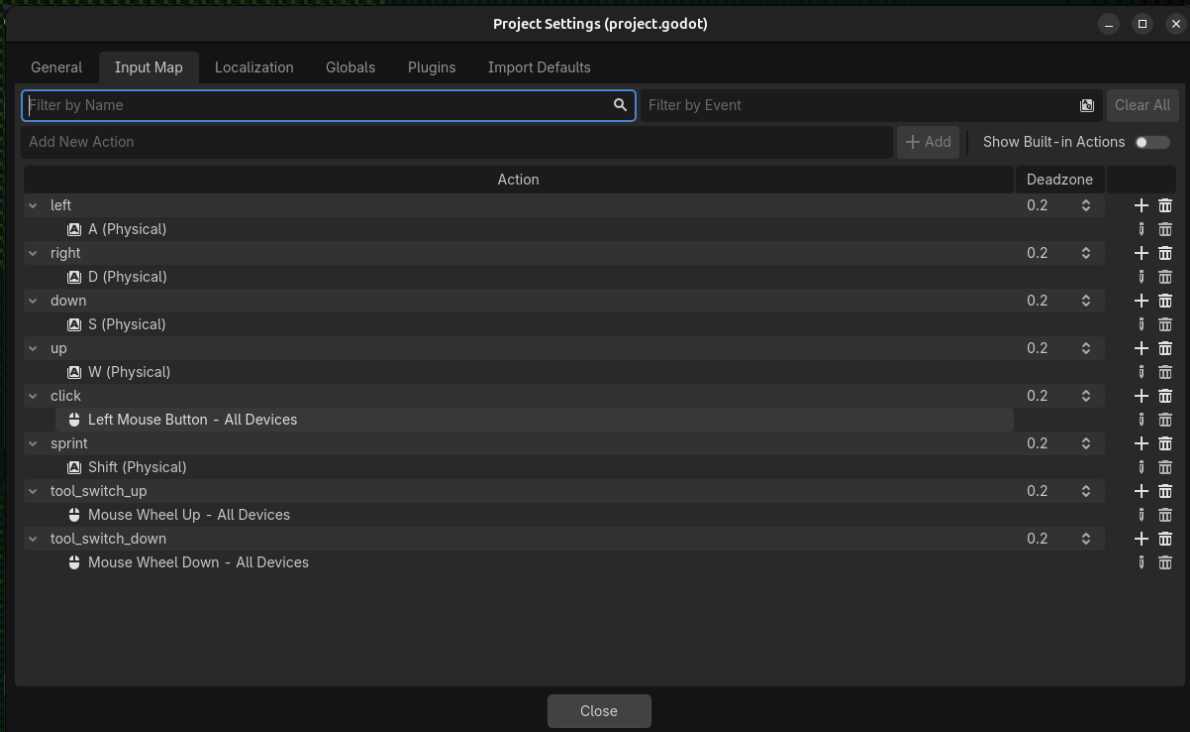
Momentan avem două animații nefolosite pentru tool-uri, dig și water.

Exercițiu: Adăugați două acțiuni pentru a schimba tool-uri. Folosiți inputul de mouse wheel up și mouse wheel down, numiți acțiunile `tool_switch_up`, `tool_switch_down`.

Mai adăugați o acțiune de și numiți-o 'click' și folosiți inputul de left mouse button.

Până o să implementăm un inventar cu interfață grafică, vom printa în consolă când schimbăm tool-urile și care este tool-ul curent.





Ce credeți că avem nevoie pentru a implementa această funcționalitate de tool switching/ usage?

Răspuns:

- Inputul adăugat de click pentru a folosi tool-ul curent
- Să stocăm tool-urile pe care le are jucătorul
- Să stocăm tool-ul curent

Enumerări(enums)

Enumerările sunt doar un grup de constante. Putem asina acelor constante valori, by default o să fie asignate valori de la 0 la N, unde N = numărul de constante create în enumerare.

Sintaxă: enum <NumeEnum> {

 NUME_CONST1

 NUME_CONST2,

 ...

}

Exercițiu: Încercați să creați o enumerare cu numele 'Tool' și cu două constate de clarate: WATER, SHOVEL.

De asemenea, creați o variabilă '_current_tool' și dați tipul Tool acelei variabile.



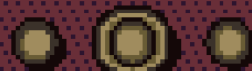
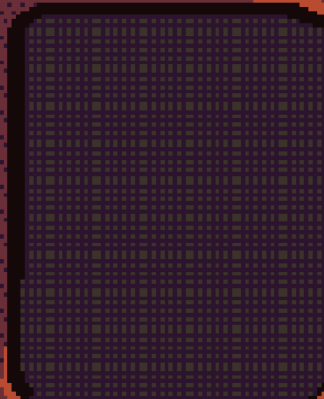
```
14  
15 ▼ enum Tool {  
16   » WATER,  
17   » SHOVEL  
18 }  
19 var _current_tool: Tool = Tool.WATER
```

Acum că avem această enumerare, putem să trecem la partea de tool switching.

Exercițiu: Creați o metodă `_switch_tool` cu un parametru `switch_direction` de tip `int` care nu returnează nimic. În corpul funcției scrieți cuvântul cheie `'pass'`, acest cuvânt cheie(keyword) este folosit pentru a indica că metoda nu este implementată încă.

În metoda `_input`, încercați să folosiți structurile de control `if` și `elif` pe acțiunile create cu `tool_switch_up`, `tool_switch_down`. În corpul `if`-ului/`elif`-ului apelați metoda `_switch_tool` creată.

Hint: folosiți `Input.is_action_pressed()`



```
29 func _input(event: InputEvent) -> void:
30     >| # preluam input de la tastatura -> returneaza un vector in 2D
31     >| _input_dir = Input.get_vector("left", "right", "up", "down")
32     >| _is_running = Input.is_action_pressed("sprint")
33     >|
34     >| if Input.is_action_pressed("tool_switch_up"):
35     >| >| _switch_tool(1)
36     >| elif Input.is_action_pressed("tool_switch_down"):
37     >| >| _switch_tool(-1)
38
39     #endregion
40
41     #region PLAYER METHODS
42     func _switch_tool(switch_direction: int) -> void:
43     >| pass
```



Pentru a implementa metoda `_switch_tool`, trebuie să modificăm tool-ul curent în funcție de inputul jucătorului.

```
41  #region PLAYER METHODS
42  func _switch_tool(switch_direction: int) -> void:
43      >| _current_tool = _current_tool + switch_direction
44      >| print(_current_tool)
45
```

Problemă: dacă dăm scroll în sus, variabila va ajunge să fie mult mai mare decât numărul de tool-uri pe care îl avem, analog dacă dăm scroll în jos. Cum putem să resetăm valoarea `_current_tool` dacă trece de numărul de elemente ale enumului `Tool`?

hint: putem accesa câte elemente are enumul prin metoda `size()`, `Tool.size()` și operatorul modulo este `%` în GDScript.

Această implementare rezolvă capătul superior, mai trebuie să rezolvăm capătul inferior. Gândiți o soluție, știm că trebuie să ignorăm valori negative și să trecem direct la ultima valoare. De exemplu dacă `_current_tool = 0` și dăm scroll jos, vrem să ajungem la `_current_tool = N` (în cazul nostru 1).

```
41  #region PLAYER METHODS
42  func _switch_tool(switch_direction: int) -> void:
43      >| _current_tool = (_current_tool + switch_direction) % Tool.size()
44      >| print(_current_tool)
45
```

Soluția este să adunăm `Tool.size()` la expresia noastră, astfel valorile nu vor putea să fie negative. Astfel de algoritmi este bine să îi testăm pe cazuri dând valori când îi creăm.

```
41 ❏ #region PLAYER METHODS
42 ▼ func _switch_tool(switch_direction: int) -> void:
43   » _current_tool = (_current_tool + switch_direction + Tool.size()) % Tool.size()
44   » print(_current_tool)
45   » # Tool.keys() returnează un array
46   » print("switched tool to:", Tool.keys()[_current_tool])
```

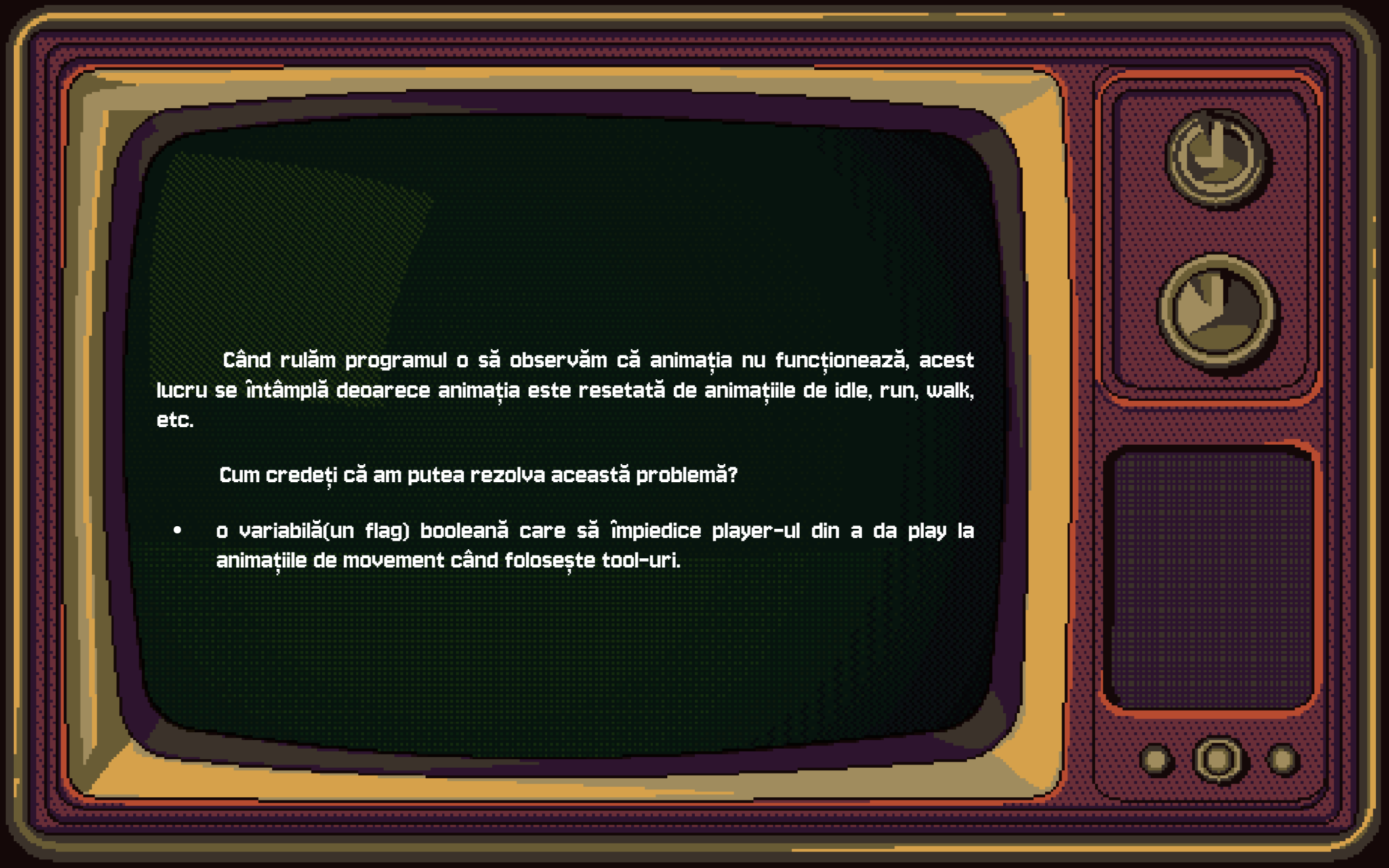
Acum că am reușit să schimbăm tool-urile, trecem la ultimul pas, trebuie să dăm play la animațiile corespunzătoare când jucătorul vrea să folosească un Tool.

Exercițiu:

Luați inputul click și dați play la animațiile corespunzătoare pentru tool. Atenție, folosiți ambele animated sprites.

Hint: Știm că putem să luăm numele tool-ului curent ca String, putem să ne folosim de această informație pentru a da play la animații. De asemenea, avem metoda `to_lower()` pentru string-uri care convertește toate caracterele uppercase în lowercase.

```
30  func _input(event: InputEvent) -> void:
31      # preluam input de la tastatura -> returneaza un vector in 2D
32      _input_dir = Input.get_vector("left", "right", "up", "down")
33      _is_running = Input.is_action_pressed("sprint")
34
35      if Input.is_action_pressed("tool_switch_up"):
36          _switch_tool(1)
37      elif Input.is_action_pressed("tool_switch_down"):
38          _switch_tool(-1)
39
40      if Input.is_action_pressed("click"):
41          var tool_anim_name: String = (Tool.keys()[_current_tool]).to_lower()
42          animated_sprite.play(tool_anim_name)
43          tools_animated_sprite.play(tool_anim_name)
44
```



Când rulăm programul o să observăm că animația nu funcționează, acest lucru se întâmplă deoarece animația este resetată de animațiile de idle, run, walk, etc.

Cum credeți că am putea rezolva această problemă?

- o variabilă(un flag) booleană care să împiedice player-ul din a da play la animațiile de movement când folosește tool-uri.



Exercițiu: creați o variabilă `_can_move` de tip `bool` și încercați să implemetați această logică pentru a da play la animații de tool-uri.

```

22  ☒ #region GODOT BUILT-IN METHODS
23  ☒ func _physics_process(delta: float) -> void:
24  ☒     ☒ if not _can_move:
25  ☒         ☒ return
26  ☒         ☒ _player_move()
27  ☒         ☒ move_and_slide()
28
29  ☒ func _process(delta: float) -> void:
30  ☒     ☒ if not _can_move:
31  ☒         ☒ return
32  ☒         ☒ _player_animate()
33
34  ☒ func _input(event: InputEvent) -> void:
35  ☒     ☒ # preluam input de la tastatura -> returneaza un vector in 2D
36  ☒     ☒ _input_dir = Input.get_vector("left", "right", "up", "down")
37  ☒     ☒ _is_running = Input.is_action_pressed("sprint")
38  ☒
39  ☒     ☒ if Input.is_action_pressed("tool_switch_up"): ☒
40  ☒
41  ☒     ☒ elif Input.is_action_pressed("tool_switch_down"): ☒
42  ☒
43  ☒
44  ☒     ☒ if Input.is_action_pressed("click"):
45  ☒         ☒ var tool_anim_name: String = (Tool.keys()[_current_tool]).to_lower()
46  ☒         ☒ animated_sprite.play(tool_anim_name)
47  ☒         ☒ tools_animated_sprite.play(tool_anim_name)
48  ☒         ☒ _can_move = false
49

```

```

#region GODOT BUILT-IN METHODS
func _physics_process(delta: float) -> void:
    if not _can_move:
        return
        _player_move()
        move_and_slide()

func _process(delta: float) -> void:
    if not _can_move:
        return
        _player_animate()

func _input(event: InputEvent) -> void:
    # preluam input de la tastatura -> returneaza un vector in 2D
    _input_dir = Input.get_vector("left", "right", "up", "down")
    _is_running = Input.is_action_pressed("sprint")

    if Input.is_action_pressed("tool_switch_up"):
    elif Input.is_action_pressed("tool_switch_down"):

    if Input.is_action_pressed("click"):
        var tool_anim_name: String = (Tool.keys()[_current_tool]).to_lower()
        animated_sprite.play(tool_anim_name)
        tools_animated_sprite.play(tool_anim_name)
        _can_move = false

```

```

#region GODOT BUILT-IN METHODS
func _physics_process(delta: float) -> void:
    if not _can_move:
        return
        _player_move()
        move_and_slide()

func _process(delta: float) -> void:
    if not _can_move:
        return
        _player_animate()

func _input(event: InputEvent) -> void:
    # preluam input de la tastatura -> returneaza un vector in 2D
    _input_dir = Input.get_vector("left", "right", "up", "down")
    _is_running = Input.is_action_pressed("sprint")

    if Input.is_action_pressed("tool_switch_up"):
    elif Input.is_action_pressed("tool_switch_down"):

    if Input.is_action_pressed("click"):
        var tool_anim_name: String = (Tool.keys()[_current_tool]).to_lower()
        animated_sprite.play(tool_anim_name)
        tools_animated_sprite.play(tool_anim_name)
        _can_move = false

```

Await

Problema la această implementare este că nu trecem înapoi la starea `_can_move` după ce se termină animația tool-ului.

Godot dispune de un cuvânt cheie 'await' care oprește execuția unei secvențe de cod până când un semnal este emis.

Sintaxă: `await <nume_componentă/nod>.<nume_semnal>`

Exercițiu: Încercați să folosiți acest mecanism pentru a reseta variabila `_can_move` pe `true` când se incheie animațiile.

Hint: `AnimatedSprite2D` dispune de un semnal numit 'animation_finished'

```
44  ▾>| if Input.is_action_pressed("click"):
45      >| >|   var tool_anim_name: String = (Tool.keys()[_current_tool]).to_lower()
46      >| >|   animated_sprite.play(tool_anim_name)
47      >| >|   tools_animated_sprite.play(tool_anim_name)
48      >| >|   _can_move = false
49      >| >|   await animated_sprite.animation_finished
50      >| >|   await tools_animated_sprite.animation_finished|
51      >| >|   _can_move = true
52
```

Setters/Getters

Trebuie să dăm flip și la animated sprite pentru tools. Știm că această nod își dă flip atunci când schimbăm valoarea variabilei `_facing_right/_facing_left`.

Godot dispune de un mecanism numit `set/get`, acest mecanism permite execuția codului atunci când schimbăm anumite variabile. Settere și gettere sunt un concept din POO care implementează proprietăți, despre care vom vorbi mai mult în următoarele cursuri.

```
1  extends CharacterBody2D
2
3  @export var _walk_speed: int = 75
4  @export var _sprint_speed: int = 100
5  var _current_speed: int = _walk_speed
6  var _is_running: bool = false
7  ▾ var _facing_left: bool = false:
8  ▾ >| set(value):
9  >| >|   animated_sprite.flip_h = value
10 >| >|   tools_animated_sprite.flip_h = value|
11
12  var _can_move: bool = true
13
```

```
1  extends CharacterBody2D
2
3  @export var _walk_speed: int = 75
4  @export var _sprint_speed: int = 100
5  var _current_speed: int = _walk_speed
6  var _is_running: bool = false
7  ▾ var _facing_left: bool = false:
8  ▾ >| set(value):
9  >| >|   animated_sprite.flip_h = value
10 >| >|   tools_animated_sprite.flip_h = value|
11
12  var _can_move: bool = true
13
```

```
1  extends CharacterBody2D
2
3  @export var _walk_speed: int = 75
4  @export var _sprint_speed: int = 100
5  var _current_speed: int = _walk_speed
6  var _is_running: bool = false
7  ▾ var _facing_left: bool = false:
8  ▾ >| set(value):
9  >| >|   animated_sprite.flip_h = value
10 >| >|   tools_animated_sprite.flip_h = value|
11
12  var _can_move: bool = true
13
```



Exercițiu: creați o metodă `_play_tool_animation` fără parametrii, care returnează void și puneți logica de tool animation în acea metodă. Apelați metoda când player-ul apasă click.

Mutați metodele de tool handling într-o altă regiune de cod.

```

43 > | if Input.is_action_pressed("tool_switch_up"):
44 > | elif Input.is_action_pressed("tool_switch_down"):
45 > |
46 > | if Input.is_action_pressed("click"):
47 > |     _play_tool_animation()
48 > | #endregion
49
50 #region TOOL METHODS
51
52 func _switch_tool(switch_direction: int) -> void:
53     _current_tool = (_current_tool + switch_direction + Tool.size()) % Tool.size() as Tool
54     print(_current_tool)
55     # Tool.keys() returnează un array
56     print("switched tool to:", Tool.keys()[_current_tool])
57
58
59 func _play_tool_animation() -> void:
60     var tool_anim_name: String = (Tool.keys()[_current_tool]).to_lower()
61     animated_sprite.play(tool_anim_name)
62     tools_animated_sprite.play(tool_anim_name)
63     _can_move = false
64     await animated_sprite.animation_finished
65     await tools_animated_sprite.animation_finished
66     _can_move = true
67 #endregion

```

```

1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int n;
6     cin >> n;
7     int a[n];
8     for (int i = 0; i < n; i++) {
9         cin >> a[i];
10    }
11    int m;
12    cin >> m;
13    int b[m];
14    for (int i = 0; i < m; i++) {
15        cin >> b[i];
16    }
17    int c[n+m];
18    for (int i = 0; i < n; i++) {
19        c[i] = a[i];
20    }
21    for (int i = n; i < n+m; i++) {
22        c[i] = b[i-n];
23    }
24    for (int i = 0; i < n+m; i++) {
25        cout << c[i] << " ";
26    }
27    return 0;
28 }

```

```

1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int n;
6     cin >> n;
7     int a[n];
8     for (int i = 0; i < n; i++) {
9         cin >> a[i];
10    }
11    int m;
12    cin >> m;
13    int b[m];
14    for (int i = 0; i < m; i++) {
15        cin >> b[i];
16    }
17    int c[n+m];
18    for (int i = 0; i < n; i++) {
19        c[i] = a[i];
20    }
21    for (int i = n; i < n+m; i++) {
22        c[i] = b[i-n];
23    }
24    for (int i = 0; i < n+m; i++) {
25        cout << c[i] << " ";
26    }
27    return 0;
28 }

```

Temă(COMPLET OPȚIONALĂ):

- Atașați un script nodului Level și numiți-l 'level'
- Creați un signal în scriptul player.gd care primește ca parametru o variabilă de tip Tool
- Emiteți semnalul la finalul animației de tool
- Conectați semnalul în script-ul level.gd(există două moduri pentru a face acest lucru)
- Creați o metodă de callback pentru semnalul conectat și printați în ea tool-ul primit
- Adăugați mai multe tool animations pe care le găsiți în folder-ul de assets

Thanks for your attention



memo-arsenal.ru